Domain-Specific Language Design

CG

November 7, 2026

Contents

1 Domain-specific languages **1**

1.1 The DSLs of the course . . . 2
1.2 Other examples . . . 2
1.3 A running example: a training DSL . . . 3
1.4 A formal definition . . . 4
1.5 External and internal DSLs . . . 4
1.6 Processing a DSL program . . . 5
1.7 Syntax, semantics and execution . . . 5
1.8 Reasons and costs . . . 6
1.9 Exercises . . . 6

2 Design decisions **6**

2.1 Syntax . . . 7
2.2 Grammar . . . 7
2.3 Architecture of the interpreter . . . 8
2.4 Abstraction level . . . 8
2.5 Types and domain constraints . . . 8
2.6 Semantics and execution . . . 10
2.7 Errors . . . 10
2.8 A natural language layer . . . 11
2.9 Evolution . . . 11
2.10 Exercises . . . 11

1 Domain-specific languages

Definition 1.1. A **domain-specific language** (DSL) is a language designed to express programs of one domain, in the vocabulary of that domain. A general-purpose language (GPL) is designed for programs of any domain.

Kaplan calls them little languages [1]. A DSL gives up generality: its programs say what a domain expert would say, and the implementation deals with how to do it. Most DSLs are not Turing complete, and that restriction makes them easier to analyze and validate than programs of a GPL. The regular languages lecture notes that regular languages are usually used to describe finite languages such as DSLs. The course has met several DSLs already, and this section starts from them.

1.1 The DSLs of the course

Dates. The first DSL of the course recognizes dates in three formats, `MM/DD/YYYY`, `DD-MM-YY` and `YYYY.MM.DD`, to validate the values of a `Date` type. Its grammar is regular, and it is written in Rascal with `lexical` rules (formal languages notes, section on regular languages). The grammar checks the shape of a date but not the calendar: day 31 of February is accepted, and rejecting it is a job for the semantics of the DSL.

LDBL. The second DSL manages the database of an image marketplace, with the commands `ADD`, `REMOVE`, `SEARCH`, `VIEW` and `HALT`, assignment of image variables, and the statements `if ... then ... else ... fi` and `loop ... do ... od`. The nesting of statements makes it context-free and not regular (formal languages notes, section on context-free languages).

SCuLPT. The Simple Cubic Language for Programming Tasks, from the first lecture, replaces typed text by tangible 3D pieces that are composed to build programs and art pieces. Unlike the two previous languages it is Turing complete. Its domain is not a kind of data but a kind of user and interface: the language changes the medium in which programs are written.

Objectilang. The language of the course project is an object-oriented language with generics. A program is made of objects, which can hold values (`val`), variables (`var`) and methods, and a `main` method. Objects extend other objects with `extends`, and generic objects declare each type parameter covariant with `+` or contravariant with `-`, as in `List[+A]`. A method is attached to an object with the `method` abstraction, as in `Cat.talk = method () : String { ... }`, and `override` replaces a method inside the construction of an object. The reserved words are `object`, `extends`, `method`, `val`, `var`, `new` and `override`, and the control flow has `if-then-else` and `for`, which iterates over a range or a collection. Objectilang is closer to a GPL than the other three, but building it touches every topic of the course: the reserved words belong to the lexer, the nesting of objects and methods to the parser, `extends` and the variance annotations to the type system (the subtyping section of the types notes uses its `Transformer[-In, +Out]` example), and method lookup and `override` to the run-time representation of objects.

1.2 Other examples

The following examples go beyond the slides.

SQL. The domain is relational data. A query says which rows are wanted, not how to find them:

```
SELECT athlete.name, workout.distance
FROM athlete
```

```
JOIN workout ON athlete.id = workout.athlete_id
WHERE workout.date = '2026-05-03';
```

There are no loops and no iteration order. Whether the join uses a hash table, nested loops or an index is decided by the database engine, and SQL engines can reorder joins and filters because the queries have no side effects.

Make. A Makefile describes a dependency graph: each target lists the files it depends on and the command that builds it. The program is the graph, not a sequence of commands; `make` orders the targets topologically and rebuilds only those whose dependencies changed.

Regular expressions. The domain is string patterns. The pattern

```
pattern = re.compile(r'[A-Z][a-z]+\d{4}')
```

matches a capitalized word followed by four digits, which would take a loop and several conditions in a GPL. The density that makes regular expressions short also makes them hard to read for someone who does not know the notation.

1.3 A running example: a training DSL

The rest of these notes use a DSL for training plans, an example that goes beyond the slides. A coach writes

```
Monday:
  run: 10km @threshold
  rest: 15min
Wednesday:
  swim: 2000m @steady
  strength: 30min
```

and the program contains nothing that is not about training. The grammar, in the BNF notation of the course, is

```
Plan      ::= Day+
Day       ::= DayName ":" Activity+
Activity  ::= Exercise ":" Quantity Intensity?
Quantity  ::= Number Unit
Intensity ::= "@" Label
DayName   ::= "Monday" | "Tuesday" | ... | "Sunday"
Exercise  ::= "run" | "swim" | "cycle" | "strength" | "rest"
Unit      ::= "km" | "m" | "min" | "h"
```

Each rule can be chosen by its first token, so the grammar is LL(1). In Rascal:

```
layout Layout = [\ \t\n\r]* !>> [\ \t\n\r];

lexical Number   = [0-9]+ !>> [0-9];
lexical Label    = [a-z]+ !>> [a-z];
lexical DayName  = "Monday" | "Tuesday" | "Wednesday" | "Thursday"
                 | "Friday" | "Saturday" | "Sunday";
```

```
lexical Exercise = "run" | "swim" | "cycle" | "strength" | "rest";
lexical Unit     = "km" !>> [a-z] | "m" !>> [a-z] | "min" !>> [a-z] | "h" !>> [a-z];

start syntax Plan = Day+ days;
syntax Day       = DayName name ":" Activity+ activities;
syntax Activity  = Exercise exercise ":" Quantity quantity Intensity? intensity;
syntax Quantity  = Number amount Unit unit;
syntax Intensity = "@" Label label;
```

1.4 A formal definition

A DSL is described by the same three components as any language:

$$DSL = \langle \Sigma, G, S \rangle,$$

where Σ is the alphabet, G a grammar over Σ that defines the valid programs, and S a semantic function from parse trees (or ASTs) to meanings. A program is a word $p \in L(G)$, and its meaning is $S(\mathit{parse}(p))$. What changes from a GPL is the size: Σ is small, G is simple, and S only has to give meaning to the concepts of the domain.

1.5 External and internal DSLs

An **external** DSL has its own syntax and its own implementation: lexer, parser, checker and interpreter or compiler. SQL, Make, the date DSL, LDBL, Objectilang and the training DSL are external. The syntax can follow the domain exactly, and programs can be read by people who do not program; the whole implementation has to be built.

An **internal** (embedded) DSL is written inside a host language, with the host's functions, method chaining or macros, and every program is also a program of the host:

```
class TrainingPlan:
    def __init__(self):
        self.days = {}
        self.current = None

    def on(self, day):
        self.current = day
        self.days[day] = []
        return self

    def run(self, km, intensity=None):
        self.days[self.current].append(('run', km * 1000, intensity))
        return self

    def rest(self, minutes):
        self.days[self.current].append(('rest', minutes, None))
        return self

plan = (TrainingPlan()
        .on('Monday')
        .run(10, intensity='threshold')
        .rest(15))
```

The parser, type checker and debugger of the host come for free. In exchange the program still looks like Python, and its errors are Python errors. Lisp and Scheme are the classic hosts for internal DSLs, because their code is a list and a macro can transform it before it runs (homoiconicity, in the runtime systems notes). Table 1 compares the two.

	External	Internal
Syntax	free, follows the domain	limited by the host
Implementation	lexer, parser, checker, evaluator	a library
Tools	built from scratch	those of the host
Error messages	in terms of the domain	in terms of the host
Typical users	domain experts	programmers

Table 1: External and internal DSLs.

1.6 Processing a DSL program

An external DSL program goes through the phases studied in the course (Figure 1). The lexer groups characters into tokens: `run: 10km @threshold` becomes the tokens `run`, `:`, `10`, `km`, `@` and `threshold`, and a character outside the alphabet is a lexical error. The parser builds the tree according to the grammar, and a sequence of tokens that the grammar does not generate is a syntax error. The semantic analysis checks what the grammar cannot express, such as the unit that each exercise accepts. The evaluator walks the checked tree and produces the result. Each kind of error is found in a different phase and deserves a different message.

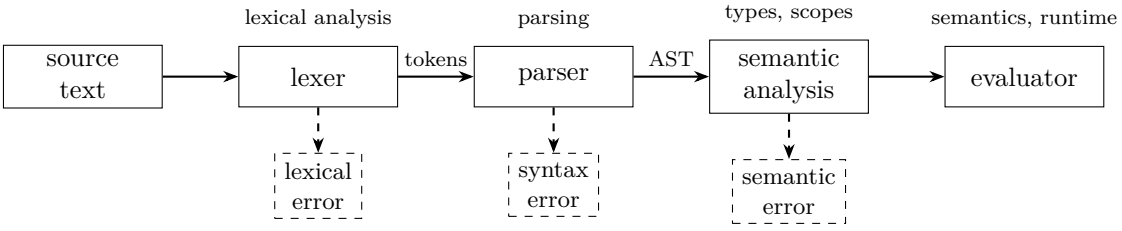


Figure 1: Phases of the processing of a DSL program, labelled with the topics of the course that study them.

1.7 Syntax, semantics and execution

A language is defined by its syntax, which says which programs are valid; its semantics, which says what they mean; and its execution model, which says how the meaning becomes action. The three styles of semantics of the course apply to a DSL as to any language. An operational rule for an activity adds it to the state σ of the schedule:

$$\langle \texttt{run:}\quad n\,u,\ \sigma\rangle \to \sigma[\mathit{activities} := \mathit{activities} \cdot \mathit{Activity}(\mathit{run}, n, u)].$$

A denotational definition maps the activity directly to a value:

$$[\![\texttt{run: 10km @threshold}]\!] = \mathit{Activity}(\mathit{run}, 10, \mathit{km}, \mathit{threshold}).$$

An axiomatic definition would state properties of plans with pre- and postconditions. The execution model can be an interpreter, which walks the AST; a compiler, which translates it to lower-level code; or a translator to another high-level language.

1.8 Reasons and costs

A DSL is worth building when its users are not programmers, as the coaches of the training DSL; when the domain has rules that the language can enforce before a program runs, as rejecting `run: 10min`; when programs should be data that can be stored, compared and transformed; and when a restricted language allows analyses that a GPL does not, as the optimization of SQL queries.

The costs are the whole implementation (lexer, parser, checker, evaluator, error messages), the lack of tools such as debuggers and editors unless they are built too, the evolution of the language without breaking old programs, and the time users need to learn the notation. When the domain is unstable or the users are programmers, a library in an existing language is often enough.

1.9 Exercises

Exercise 1.2. Classify the date DSL, LDBL and the training DSL as regular or context-free.

Solution. The date DSL is regular: every rule is a concatenation of finite sets of symbols. LDBL is context-free and not regular, because `if` and `loop` contain commands that can contain other `if`s and `loop`s. The training DSL has no nesting: a plan is a sequence of days and a day a sequence of activities, so the language is regular, even though it is written with a context-free grammar.

Exercise 1.3. In Objectilang, `object`, `extends` and `override` are reserved words. Which phase rejects a program that uses `extends` as the name of a value?

Solution. The lexical definition of identifiers excludes the reserved words, so `extends` is never an identifier, and the parser finds the keyword where the grammar expects a name: it is a syntax error. The LDBL grammar of the formal languages notes shows the mechanism in Rascal, with a `keyword` declaration and a reject in the lexical of names.

Exercise 1.4. Is the training DSL external or internal? Write the plan `Monday: run: 10km @threshold rest: 15min` with the internal version in Python.

Solution. The training DSL is external: it has its own grammar and its own parser. The internal version is `TrainingPlan().on('Monday').run(10, intensity='threshold').rest(15)`, which is the program tested in `internal.py`.

2 Design decisions

Building a DSL is a sequence of decisions about syntax, grammar, types, semantics, execution and errors, and the decisions interact. A more expressive language is harder to implement, analyze and maintain; a more restricted one leaves users without a way to say some things. This section goes beyond the slides, and uses the training DSL of Section 1 as its example.

2.1 Syntax

Who writes the programs. Programmers accept braces, semicolons and type annotations in exchange for a familiar structure. Domain experts need a notation that reads like their domain. The users of the training DSL are coaches and athletes, so its syntax follows the way they describe a session.

Declarative and imperative. A declarative program says what exists; an imperative program says what to do. The training plan `Monday: run: 10km @threshold` describes a session. The internal Python version of Section 1 gives the same information as a sequence of calls, `.on('Monday').run(10, intensity='threshold')`, that build the plan step by step. The declarative version talks about the session, not about the operations that record it.

Verbosity. A verbose syntax is explicit and easy to read the first time:

```
exercise type=run distance=10 unit=km intensity=threshold
```

A concise syntax has to be learned but is faster to write:

```
run: 10km @threshold
```

If programs are written once and read many times, readability matters more; if the same users write programs every day, conciseness does. The training DSL is concise.

Layout. When indentation has meaning, as in Python, the lexer has to produce tokens for the start and end of each indented block. When blocks are delimited explicitly, as in C, the lexer can discard all whitespace. The Rascal grammar of the training DSL takes the second option: its layout discards spaces and newlines, and day names start new blocks, so the indentation of the example is only a convention for the reader.

2.2 Grammar

Ambiguity. A grammar is ambiguous when a word has two parse trees (formal languages notes). In a DSL an ambiguity means that a program can mean two things. If the grammar allowed a second qualifier after the intensity, in

```
run 10 km @threshold easy
```

the word `easy` could modify the intensity or the whole activity. The grammar has to decide, with separators, with different tokens for different qualifiers, or with rules that allow only one reading.

Context-free is enough. The syntax of most DSLs is context-free, and with an LL(1) or LR(1) grammar it is parsed in linear time. Context-sensitive conditions, such as the unit that an exercise accepts, do not belong in the grammar but in semantic analysis, after the tree is built.

LL or LR. An LL(1) grammar can be parsed by recursive descent, one function per nonterminal, and the error messages can be precise; it has to be left-factored and without left recursion. LR grammars cover more languages, but their tables are built by generators such as Yacc, Bison or the

parser generator of Rascal. For a small DSL written by hand, LL(1) is the usual choice (parsing notes). The grammar of the training DSL is LL(1).

Error recovery. A parser that stops at the first error makes the user fix one error at a time. Panic mode, from the parsing lecture, skips tokens until a synchronization token (a new day name, in the training DSL) and continues from there. The message is part of the design:

```
line 2: invalid unit 'min' for exercise 'run'
```

says more than `parse error at token 'min'`.

2.3 Architecture of the interpreter

The decisions meet in the implementation (Figure 2). Each component has one responsibility: the lexer does not know the grammar, the parser does not know the rules of the domain, and the checker does not know how the plan is executed. Each component can be tested on its own, and every error is reported by the component that found it.

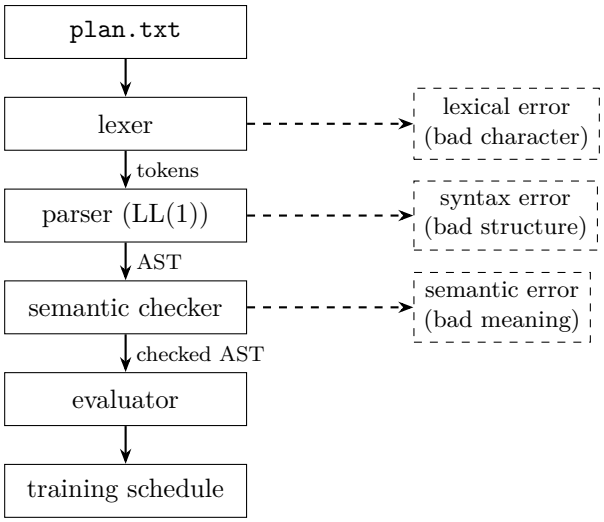


Figure 2: Architecture of an interpreter for the training DSL.

2.4 Abstraction level

A DSL should use the abstractions of its domain, no higher and no lower (Figure 3). A language at the level of a GPL makes the user think as a programmer. Natural language loses precision and needs a translation back into a structured form. When a DSL makes its user think about the level below, the abstraction leaks: an error message such as `NullPointerException in ActivityBuilder.java:43` speaks about the implementation, which a coach does not know.

2.5 Types and domain constraints

Programs that respect the grammar can still be meaningless: `run:  10min` gives a duration to an exercise measured in distance, and `rest:  5km` the opposite. A type system for the DSL rejects

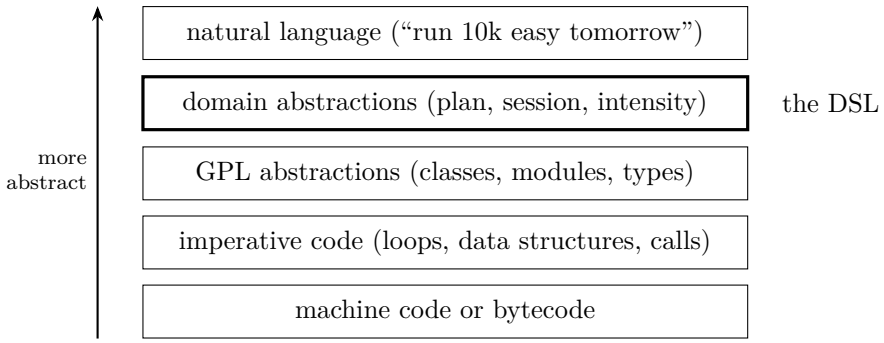


Figure 3: Levels of abstraction. The DSL is at the level of the domain.

them before the program runs. The type of each exercise is known from its name: `run` has type *Distance* → *Activity*, and `rest` has type *Duration* → *Activity*, so the user never writes a type annotation. The checker of the training DSL implements these rules, and also requires positive amounts:

```
set[str] distanceExercises = {"run", "swim", "cycle"};
set[str] distanceUnits = {"km", "m"};
set[str] durationUnits = {"min", "h"};

list[str] check(Plan p) {
  list[str] errors = [];
  for (/Activity a := p) {
    str ex = "<a.exercise>";
    str unit = "<a.quantity.unit>";
    set[str] expected = ex in distanceExercises ? distanceUnits : durationUnits;
    if (unit notin expected) {
      errors += "line <a.src.begin.line>: invalid unit \'<unit>\' for exercise \'<ex
    >\'";
    }
    if (toInt("<a.quantity.amount>") == 0) {
      errors += "line <a.src.begin.line>: the amount of \'<ex>\' must be positive";
    }
  }
  return errors;
}
```

The tests check the messages for `run: 10min`, `rest: 5km` and `run: 0km`. A negative amount, `run: -5km`, does not even reach the checker: the grammar has no minus sign, so it is a syntax error.

Constraints of this kind can be written as types with a predicate, called refinement types:

$$\mathit{Distance} = \{d \in \mathbb{N} \mid d > 0\}.$$

The constraints are checked statically, when the program is analyzed. A plan is stored and used weeks after it is written, and a dynamic check would only find the error when the plan runs. The rule for the training DSL is to reject a wrong plan before it is stored.

2.6 Semantics and execution

Section 1 gave an operational and a denotational rule for one activity. For a whole plan, the denotation of a plan is the set of denotations of its days,

$$[\![plan]\!] = \{[\![d]\!] \mid d \in days(plan)\},$$

and two programs with the same denotation mean the same, even if they are written differently. A simple evaluator walks the checked tree and computes, for instance, the total distance of the plan:

```
int meters(Quantity q) = "<q.unit>" == "km" ? toInt("<q.amount>") * 1000 : toInt("<q.
    amount>");

int totalMeters(Plan p) {
  int total = 0;
  for (/Activity a := p, "<a.exercise>" in distanceExercises) {
    total += meters(a.quantity);
  }
  return total;
}
```

For the example plan the result is 12000 meters, 10 km of running and 2000 m of swimming.

The execution model can be an interpreter, a compiler or a translator to another high-level language (Figure 4). For a DSL an interpreter is the usual first choice: there is no build step, and errors point to the source. Compiling pays off when the domain involves heavy computation, and translating to another language when the programs must run in an existing ecosystem, as TypeScript translated to JavaScript. The training DSL produces schedules and totals, and an interpreter is enough.

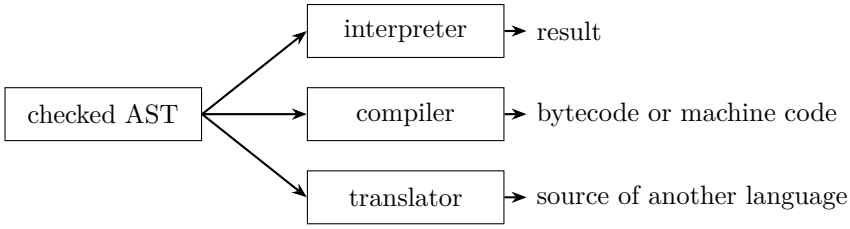


Figure 4: Three execution models for a DSL.

2.7 Errors

Errors in a DSL come in four levels, each found by a different check:

1. lexical: `r%n:  10km` has a character outside the alphabet;
2. syntactic: `run 10km:` has valid tokens in the wrong order;
3. semantic: `run:  10min` has a valid structure and the wrong unit;
4. domain: `run:  500km @sprint` is valid and well typed, but no session can have it.

A message should say what is wrong, where, and what was expected, in the terms of the domain.

2.8 A natural language layer

The training DSL was designed to accept natural language input, such as "add a threshold run on Monday, about 10k", translated into a DSL program by a language model before the lexer (Figure 5). The layer is not part of the DSL. The DSL acts as a structured intermediate representation: the model does not produce schedules but DSL programs, which go through the whole pipeline. If the program has a wrong unit, an impossible value or a broken structure, the checker rejects it, and its message can be sent back to the model to ask for a correction. The model deals with the ambiguity of natural language, and the DSL with structure and correctness.

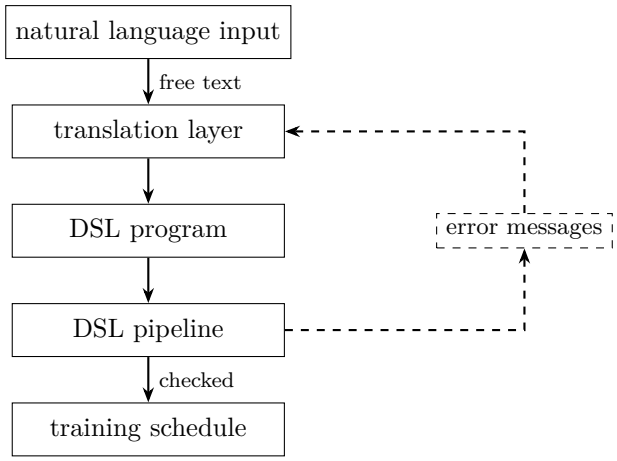


Figure 5: A natural language layer in front of the DSL.

2.9 Evolution

A DSL whose changes break the programs already written is a liability, and stored plans are exactly such programs. New terminals, such as a new exercise, and new optional constructs do not affect programs that do not use them. A new required construct breaks old programs, and needs a version: a header such as `version: 2` tells the parser which grammar to use.

Table 2 collects the decisions taken for the training DSL.

2.10 Exercises

Exercise 2.1. Add `cycle: 2h` to a plan. Does the checker accept it? Which change to the checker would accept both distances and durations for `cycle`?

Solution. The checker rejects it: `cycle` is in `distanceExercises`, so it expects `km` or `m`, and the message is `invalid unit 'h' for exercise 'cycle'`. Accepting both needs a third set, for exercises that accept any unit, and for those the expected units are the union of the two sets. The total distance would then have to skip the activities of `cycle` measured in hours.

Exercise 2.2. Classify each error: `Monday: run: 10km @Threshold`, `Monday: run: 10min`, `Monday run: 10km`, `Monday: run: 90km`.

Decision	Choice	Reason
Style	declarative	coaches describe sessions
Syntax	concise	the same users write plans every day
Layout	ignored by the grammar	simpler lexer; day names start blocks
Parser	LL(1)	small grammar, precise errors
Types	inferred, static	no annotations, errors before storing
Execution	interpreted	no performance requirement
Errors	in terms of the domain	users are not programmers
Natural language	separate layer	the semantics of the DSL stays simple
Evolution	additive changes	stored plans keep working

Table 2: Design decisions for the training DSL.

Solution. `@Threshold` is a syntax error: `Label` only has lowercase letters, so `T` cannot start the label, and Rascal reports the position. `run: 10min` is a semantic error (wrong unit). The missing colon after `Monday` is a syntax error. `run: 90km` passes every check of the implementation; it is a domain error that a rule on the maximum distance of a session would catch.

Exercise 2.3. The example of the Objectilang statement declares `List[+A]` and assigns a value of type `List[Dog]` to a value declared with type `List[Animal]`. Which phase checks the assignment, and why is it valid?

Solution. The type checker, in the semantic analysis. With `Dog` <: `Animal` and `A` covariant, `List[Dog]` <: `List[Animal]`, and by subsumption a value of the subtype can be used where the supertype is expected (subtyping section of the types notes).

References

[1] Randy M. Kaplan. *Constructing Language Processors for Little Languages.* Wiley, 1994.